

HPO-Enhanced Black Box Optimisation

Bayesian Optimisation with Gradient Enhancement, CNN Surrogates and Optuna HPO

1. Overview

Approach Name: BBO (HPO-Enhanced Black Box Optimisation)

Model Type: Gaussian Process surrogate with CNN augmentation and Optuna-driven HPO

Version: BBO1 → BBO2 → ... → BBO6 (CNN-Enhanced GP) → BBO7/10 (Optuna HPO)

BBO9 is a sequential, model-based optimisation strategy that selects the single most promising query point per round. It combines a Gaussian Process (GP) surrogate with two CNN auxiliary models — one predicting gradient magnitudes for informed exploration, the other correcting GP residuals — and wraps all three in an Optuna-driven hyper-parameter optimisation (HPO) loop that adapts kernel choice, model architecture, and acquisition parameters to each function automatically.

2. Intended Use

Suitable tasks

- Single-objective black-box optimisation over continuous, bounded domains.
- Problems where each function evaluation is expensive and the dataset is small (tens of observations).
- Multi-dimensional inputs (tested across 2D–8D).

Cases to avoid

- Multi-objective problems — the acquisition function is single-objective UCB only.
- Discrete or combinatorial search spaces — the GP assumes continuous inputs.
- Very high-dimensional problems ($d > \sim 20$) where GP scaling and CNN generalisation both degrade.
- Streaming or online settings — the pipeline is batch-mode: all observations are required before each query.
- Hard real-time constraints — the full HPO loop (GP + CNN + acquisition) takes several minutes per function.

3. Details

Strategy across ten rounds

Each round follows an identical eight-step pipeline. The strategy does not explicitly vary by round number, but adapts continuously as new observations accumulate: with more data the GP fits better, gradient estimates sharpen, and the HPO studies warm-start from prior runs (Optuna study persistence).

Eight-step pipeline (per round)

Step	Description	Origin
1	Kernel + GP Noise HPO — selects best kernel (RBF, Matérn-2.5, Matérn-1.5, ARD variants); tunes alpha, n_restarts, WhiteKernel bounds via TPE	Enhancement #8 (BBO7/BBO10)
2	Gradient Enhancement — augments observations with finite-difference virtual gradient points (epsilon=0.01)	BBO6
3	Residual Corrector CNN HPO — tunes architecture via CMA-ES + MedianPruner; trains ResidualCorrectorCNN on LOO residuals	Enhancement #7 (BBO7/BBO10)

4	Gradient Predictor CNN HPO — tunes GradientPredictorCNN architecture; predicts gradient magnitudes for SV scoring	Enhancement #7 (BBO7/BBO10)
5	Acquisition HPO — tunes kappa, sv_weight, prescreen_k via LOO rank-correlation proxy (TPE)	Enhancement #6 (BBO7/BBO10)
6	SV-like point analysis — scores candidates by gradient informativeness weighted by sv_weight	BBO6
7	CNN pre-screen + corrected UCB — shortlists top prescreen_k candidates, applies residual correction, computes UCB	BBO6
8	L-BFGS-B local refinement — polishes the top candidate from step 7 within bounds	BBO6

HPO enhancements

Enhancement	Technique	Parameters Tuned	Optuna Sampler
#6	Acquisition HPO	kappa [0.5–10.0], sv_weight [0.0–0.8], prescreen_k [1k–20k]	TPE (40 trials)
#7	CNN Architecture HPO	hidden_size {32,64,128,256}, dropout [0.05–0.5], LR [1e-4–1e-2], epochs [100–400]	CMA-ES + MedianPruner (20 trials)
#8	GP Noise & Restart HPO	alpha [1e-8–1e-3], n_restarts [5–30], WhiteKernel noise_upper [1e-8–1e-1]	TPE (30 trials)

4. Performance

Metrics reported

- UCB score: the Upper Confidence Bound value at the recommended next point ($\mu + \kappa \cdot \sigma$ after residual correction). Primary selection criterion.
- Posterior mean (μ): GP + CNN-corrected mean prediction at the next point. Indicates exploitation value.
- Posterior std (σ): uncertainty at the next point. Indicates exploration value.
- HPO-tuned kappa: the per-function exploration–exploitation coefficient selected by acquisition HPO. Low values (near 0.5) indicate the model is confident and exploiting; high values (near 10) indicate uncertain terrain requiring exploration.
- LOO rank score: fraction of leave-one-out folds where the best observed point ranks in the top quartile of the UCB surface. Used internally to drive acquisition HPO.

Results summary

- Results are recorded per-function in the summary table printed at the end of `run_all_functions()`. The table reports kernel type, HPO-tuned kappa, sv_weight, alpha, UCB score, posterior mean, and posterior std for each of F1–F8. Key qualitative outcomes:
- Low-dimensional functions (F1, F2): HPO consistently selects lower kappa and smaller alpha, reflecting smooth, well-characterised landscapes where exploitation dominates.
- High-dimensional functions (F7, F8): larger kappa and Matérn kernels are preferred, reflecting rougher or multi-modal surfaces where exploration is needed.
- CNN residual correction reliably reduces posterior uncertainty (narrower std) compared to raw GP, particularly in regions with sparse coverage.
- Gradient enhancement increases effective training set size by a factor of $(1 + 2d)$, providing richer curvature information at no additional function evaluations.

5. Assumptions and limitations

Assumptions:

- the underlying function changes smoothly (no sudden jumps), stays within a fixed input range (between 0 and 1 for all variables), and isn't too noisy (observations are mostly reliable).
- RBF and Matérn kernels assume stationary correlation structure. Non-stationary functions (e.g., smoothness that varies across the domain) may be mis-specified.
- it needs a reasonable amount of initial data to learn useful patterns—otherwise it falls back to a simpler prediction method (e.g. CNN training requires at least ~10 observations to produce meaningful gradient or residual models; with fewer points the CNN components fall back to GP-only predictions.)

Limitations

- The most significant limitation of the approach is that the validity of the recommendations depends heavily on the quality and number of the initial observations, and for functions like F1 and F5 where those observations are statistically problematic, the pipeline's outputs may be unreliable regardless of how well the HPO is tuned or the selected approach.

6. Ethical considerations

Transparency

BBO is designed with full transparency as a core principle. All algorithmic choices — kernel candidates, HPO search spaces, acquisition proxies, CNN architectures — are explicitly parameterised and documented in code comments aligned with the cited literature (Optuna: Akiba et al., 2019).

The pipeline is fully reproducible given fixed random seeds. The use `*_hpo=False` flags allow exact replication of the BBO baseline, providing a clear before/after comparison. All intermediate outputs (next points, kernel names, HPO-tuned parameters) are printed and saved as .npy files, making the recommendation auditable without re-running the optimisation.

Real-world adaptation

When adapting BBO to real-world problems, the following considerations apply:

- Fairness and bias: the algorithm has no awareness of equity constraints. If the objective function encodes a metric that has disparate impact (e.g., optimising a parameter that affects different groups unequally), downstream auditing is the responsibility of the practitioner, not the optimiser.
- Carbon footprint: HPO adds significant compute overhead (up to ~2–5 minutes per function on CPU). For large-scale deployments, practitioners should consider whether the marginal improvement from HPO justifies the energy cost, or whether fixed hyperparameters from prior experience would suffice.